

02_entorno-python-backend

#linux #bash #python #fastapi #backend #venv #pytest #PostgreSQL #SIEM

1 Objetivo de la nota

Esta nota analiza el entorno Python utilizado por el backend del laboratorio SIEM MVP.

El objetivo es entender cómo está organizada la parte backend, qué archivos forman parte del entorno de ejecución, qué elementos son código fuente y qué elementos son generados localmente durante el desarrollo.

El backend es la capa encargada de exponer la API REST, conectar con PostgreSQL, validar datos, aplicar modelos ORM, ejecutar migraciones y permitir pruebas automatizadas.

2 Ubicación del backend

El backend se encuentra en:

```
siem-lab/backend/
```

Desde la raíz del proyecto:

```
cd ~/siem-lab
```

se puede listar su contenido con:

```
ls -la backend
```

La salida muestra:

```
backend/
├── alembic/
├── alembic.ini
├── app/
├── Dockerfile
├── .pytest_cache/
├── pytest.ini
├── requirements.txt
├── tests/
└── venv/
```

Esto indica que el backend está preparado tanto para desarrollo local como para ejecución con Docker.

3 Papel del backend dentro del laboratorio

El backend es la capa principal de lógica del proyecto.

Sus responsabilidades son:

- Exponer endpoints HTTP mediante FastAPI.
- Validar datos de entrada con Pydantic.
- Conectar con PostgreSQL mediante SQLAlchemy.
- Definir modelos ORM.
- Gestionar migraciones con Alembic.
- Crear reglas, eventos y alertas.
- Consultar métricas.
- Permitir pruebas automatizadas con pytest.

Relación general:

```
Frontend / curl / Swagger
    ↓
FastAPI backend
    ↓
Schemas Pydantic
    ↓
Modelos SQLAlchemy
    ↓
PostgreSQL
```

4 Estructura principal del backend

La estructura del backend es:

```
backend/
├─ alembic/
├─ alembic.ini
├─ app/
├─ Dockerfile
├─ pytest.ini
├─ requirements.txt
├─ tests/
└─ venv/
```

Cada elemento tiene una función concreta:

```
app/
    ↓
código principal de la aplicación

alembic/
    ↓
migraciones de base de datos

alembic.ini
    ↓
configuración de Alembic

Dockerfile
    ↓
construcción de imagen Docker del backend

requirements.txt
    ↓
dependencias Python

pytest.ini
    ↓
configuración de pytest

tests/
    ↓
pruebas automatizadas

venv/
    ↓
entorno virtual local, no entregable
```

5 Directorio app/

La carpeta:

```
backend/app/
```

contiene el código real de la aplicación.

Estructura mostrada:

```
backend/app/  
├─ api/  
├─ db/  
├─ __init__.py  
├─ main.py  
├─ models/  
├─ __pycache__/  
└─ schemas/
```

Esta carpeta es el núcleo del backend.

La separación por responsabilidades es:

```
api/  
  ↓  
  rutas HTTP  
  
db/  
  ↓  
  conexión y sesiones de base de datos  
  
models/  
  ↓  
  modelos ORM de SQLAlchemy  
  
schemas/  
  ↓  
  schemas Pydantic de entrada y salida  
  
main.py  
  ↓  
  punto de entrada de FastAPI
```

6 Archivo `main.py`

El archivo:

```
backend/app/main.py
```

es el punto de entrada de la aplicación FastAPI.

Su función es:

```
- Crear la instancia app = FastAPI(...).  
- Configurar CORS.  
- Registrar los routers.  
- Dejar disponibles los endpoints del backend.
```

Relación:

```
main.py  
  ↓  
FastAPI app  
  ↓  
include_router(...)
```

↓
endpoints disponibles

Este archivo conecta todos los módulos de la API.

7 Carpeta `api/`

La carpeta:

`backend/app/api/`

contiene las rutas de la API.

En concreto, dentro de:

`backend/app/api/routes/`

aparecen:

```
alerts.py
events.py
health.py
info.py
ingest.py
metrics.py
rules.py
```

Función de cada archivo:

```
health.py
  ↓
endpoint de comprobación de salud

info.py
  ↓
información de versión y build

events.py
  ↓
consulta de eventos

ingest.py
  ↓
ingesta de eventos y generación de alertas

rules.py
  ↓
creación y listado de reglas

alerts.py
  ↓
consulta y actualización de alertas

metrics.py
  ↓
métricas agregadas del sistema
```

Esta carpeta representa la capa HTTP del backend.

8 Carpeta `db/`

La carpeta:

```
backend/app/db/
```

contiene:

```
base.py
database.py
session.py
```

Función de cada archivo:

```
base.py
↓
define la clase Base para los modelos SQLAlchemy

database.py
↓
crea engine, SessionLocal y test_db_connection()

session.py
↓
define get_db() para inyectar sesiones en endpoints
```

Relación:

```
DATABASE_URL
↓
create_engine()
↓
SessionLocal
↓
get_db()
↓
endpoints FastAPI
```

Esta carpeta permite que la API trabaje con PostgreSQL.

9 Carpeta models/

La carpeta:

```
backend/app/models/
```

contiene:

```
alert.py
event.py
rule.py
```

Estos archivos definen los modelos ORM.

Cada modelo representa una tabla de PostgreSQL:

```
event.py
↓
tabla events

rule.py
↓
tabla rules

alert.py
```

↓
tabla alerts

La relación principal del laboratorio es:

Event
↓
se evalúa contra

Rule
↓
si coincide genera

Alert

Estos modelos son la base de la persistencia del sistema.

10 Carpeta schemas/

La carpeta:

backend/app/schemas/

contiene:

alert.py
event.py
ingest.py
rule.py

Los schemas están definidos con Pydantic.

Su función es:

- Validar datos recibidos por la API.
- Definir modelos de respuesta.
- Controlar tipos y restricciones.
- Convertir objetos ORM en JSON.

Relación típica:

JSON de entrada
↓
Schema Pydantic
↓
Modelo SQLAlchemy
↓
Base de datos
↓
Schema de salida
↓
JSON de respuesta

1 1 Archivo requirements.txt

El archivo:

backend/requirements.txt

define las dependencias Python del backend.

Su función es permitir reconstruir el entorno sin entregar el `venv`.

El flujo normal sería:

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Este archivo sí debe incluirse en la entrega porque permite instalar las dependencias necesarias.

1 2 Entorno virtual `venv/`

En el backend existe:

```
backend/venv/
```

Este directorio contiene el entorno virtual local de Python.

En la salida aparecen archivos como:

```
backend/venv/bin/activate
backend/venv/bin/pip
backend/venv/bin/uvicorn
backend/venv/pyvenv.cfg
```

Esto indica que el backend puede ejecutarse localmente desde un entorno Python aislado.

Ventajas del `venv`:

- Aísla dependencias del sistema.
- Evita conflictos con otros proyectos.
- Permite instalar versiones concretas de paquetes.

Pero para la entrega:

```
venv/ no debe incluirse
```

Motivo:

- Es pesado.
- Es recreable.
- Depende de la máquina local.
- No es código fuente.

Lo correcto es entregar `requirements.txt`, no el entorno virtual.

1 3 Archivo `Dockerfile`

El archivo:

```
backend/Dockerfile
```

define cómo construir la imagen Docker del backend.

Su función es permitir que el backend se ejecute dentro de un contenedor.

Relación:

```
backend/Dockerfile
↓
imagen de backend
```

↓
servicio api en docker/compose.yml

Este archivo forma parte del proyecto y sí debe entregarse.

Docker permite que el backend se ejecute de forma más reproducible que en un entorno local manual.

1 4 Alembic

El backend contiene:

```
backend/alembic/  
backend/alembic.ini
```

Alembic se usa para gestionar migraciones de base de datos.

Dentro de:

```
backend/alembic/versions/
```

aparecen migraciones como:

```
create_events_table  
add_meta_to_events  
add_rules_and_alerts  
add_group_key_to_alerts  
add_throttle_to_rules  
add_threshold_to_rules  
add_status_and_updated_at_to_alerts  
fix_group_key_default
```

Esto demuestra que la base de datos evolucionó de forma progresiva.

Relación:

```
Modelos SQLAlchemy  
    ↓  
Migraciones Alembic  
    ↓  
Cambios en PostgreSQL
```

Alembic permite versionar la estructura de la base de datos.

1 5 Tests con pytest

El backend incluye:

```
backend/tests/  
backend/pytest.ini
```

Archivos detectados:

```
backend/tests/test_alerts_ui.py  
backend/tests/test_health.py
```

Esto indica que existen pruebas automatizadas.

El archivo:

```
backend/pytest.ini
```

sirve para configurar pytest.

Las pruebas encontradas apuntan a validar:

- Endpoint de health.
- Endpoints de alertas UI.

Este apartado se analizará con más detalle en el módulo de pruebas y validación.

16 Cachés generadas por Python

En la salida aparecen varios elementos generados automáticamente:

```
__pycache__/  
*.pyc  
.pytest_cache/
```

Ejemplos:

```
backend/app/__pycache__/main.cpython-312.pyc  
backend/app/models/__pycache__/alert.cpython-312.pyc  
backend/.pytest_cache/  
backend/tests/__pycache__/
```

Estos archivos se generan al ejecutar Python o pytest.

No forman parte del código fuente.

Para la entrega deben excluirse:

```
__pycache__/  
*.pyc  
.pytest_cache/
```

17 Relación entre backend local y Docker

El backend puede entenderse en dos modos de ejecución.

Modo local

Usa:

```
backend/venv/  
requirements.txt  
uvicorn  
DATABASE_URL
```

Flujo conceptual:

```
activar venv  
↓  
instalar dependencias  
↓  
configurar variables  
↓  
ejecutar uvicorn  
↓  
FastAPI disponible
```

Modo Docker

Usa:

```
backend/Dockerfile
docker/compose.yml
.env.example / .env
```

Flujo conceptual:

```
docker compose
  ↓
construye imagen backend
  ↓
levanta API
  ↓
conecta con PostgreSQL
```

Para una entrega académica, Docker es más fácil de reproducir.

1 8 Relación con PostgreSQL

El backend no guarda datos en memoria.

Trabaja con PostgreSQL mediante:

```
SQLAlchemy
DATABASE_URL
SessionLocal
get_db()
```

Flujo:

```
Endpoint FastAPI
  ↓
Depends(get_db)
  ↓
Session SQLAlchemy
  ↓
Modelo ORM
  ↓
PostgreSQL
```

Esto afecta a todos los módulos principales:

```
events
rules
alerts
metrics
health
```

1 9 Relación con FastAPI

FastAPI actúa como framework principal del backend.

Permite:

- Definir endpoints con decoradores.
- Inyectar dependencias con Depends.
- Validar datos con Pydantic.
- Generar documentación Swagger.
- Devolver respuestas JSON.

Relación:

```
main.py
  ↓
FastAPI

routes/
  ↓
endpoints

schemas/
  ↓
validación

models/
  ↓
persistencia
```

Swagger se consulta desde:

```
http://localhost:8000/docs
```

2.0 Relación con el frontend

El frontend consume el backend mediante HTTP.

La URL base configurada en el frontend es:

```
http://localhost:8000
```

Esto significa que el backend debe estar levantado en el puerto 8000.

Flujo:

```
frontend/assets/app.js
  ↓
apiFetch()
  ↓
GET /alerts
  ↓
backend FastAPI
  ↓
respuesta JSON
  ↓
tabla HTML
```

Por tanto, el backend es la fuente de datos de la interfaz.

2.1 Archivos del backend que sí forman parte de la entrega

Deben incluirse:

```
backend/app/
backend/alembic/
backend/alembic.ini
backend/Dockerfile
backend/requirements.txt
backend/pytest.ini
backend/tests/
```

Motivo:

```
app/
  ↓
código principal

alembic/
  ↓
migraciones

Dockerfile
  ↓
contenedor backend

requirements.txt
  ↓
dependencias

pytest.ini + tests/
  ↓
validación automatizada
```

2 2 Archivos del backend que no deben entregarse

Deben excluirse:

```
backend/venv/
backend/.pytest_cache/
backend/**/__pycache__/*
backend/**/*.*pyc
```

Motivo:

```
venv/
  ↓
entorno local recreable

.pytest_cache/
  ↓
caché de pruebas

__pycache__/*
  ↓
caché de Python

*.pyc
  ↓
bytecode generado
```

Esto mantiene la entrega limpia.

2 3 Comandos útiles del entorno Python

Entrar al backend:

```
cd ~/siem-lab/backend
```

Crear entorno virtual:

```
python3 -m venv venv
```

Activar entorno virtual:

```
source venv/bin/activate
```

Instalar dependencias:

```
pip install -r requirements.txt
```

Ejecutar backend localmente, si las variables y base de datos están disponibles:

```
uvicorn app.main:app --reload
```

Ejecutar tests:

```
pytest
```

Ver archivos Python principales:

```
find app -maxdepth 4 -type f | sort
```

Buscar cachés generadas:

```
find . -type d -name "__pycache__" -print
find . -type f -name "*.pyc" -print
find . -type d -name ".pytest_cache" -print
```

2.4 Resumen técnico

El entorno Python del backend está organizado para permitir desarrollo local, ejecución con Docker, migraciones y pruebas.

La estructura principal es:

```
backend/
├─ app/
│   ├── api/
│   ├── db/
│   ├── models/
│   └─ schemas/
├─ alembic/
├─ tests/
├─ Dockerfile
├─ requirements.txt
└─ pytest.ini
```

El backend combina:

```
Python
FastAPI
Pydantic
SQLAlchemy
Alembic
PostgreSQL
pytest
Docker
```

Su función dentro del laboratorio es exponer la API, gestionar reglas, recibir eventos, generar alertas y permitir que el frontend consulte y actualice el estado de esas alertas.

El entorno local contiene elementos útiles para desarrollo, como `venv/`, `__pycache__/` y `.pytest_cache/`, pero estos no deben incluirse en la entrega final.